Sprawozdanie z projektu
Symulator stanowiska dyspozytorskiego (SCADA)

Tomasz Nazar 197613, Filip Pietrzak 198275

1 kwietnia 2026

1 Sformułowanie zadania i założenia szczegółowe

Celem projektu było stworzenie aplikacji symulującej stanowisko dyspozytorskie (typu SCADA) nadzorujące wirtualną linię produkcyjną. Program musiał łączyć elementy diagnostyki nadzorowanego procesu produkcji (zależne od rzeczywistych parametrów sprzętowych komputera) oraz autodiagnostyki operatora (tzw. mechanizm czuwaka). Aplikacja została zrealizowana w języku Python z użyciem biblioteki PyQt6.

1.1 Założenia diagnostyki procesu

Zdecydowano, że głównym parametrem napędzającym symulację będzie rzeczywiste obciążenie procesora (CPU) komputera uruchamiającego aplikację. Na jego podstawie wyliczana jest wirtualna temperatura silnika linii produkcyjnej. Wprowadzono następujące progi i reakcje automatyczne:

- **Przekroczenie 85°C:** Automatyczne włączenie dodatkowego wentylatora chłodzącego.
- **Spadek poniżej 60°C:** Wyłączenie wentylatora.
- **Krytyczne przekroczenie 100°C:** Awaryjne zredukowanie prędkości linii produkcyjnej do 50%.
- **Ustabilizowanie poniżej 80°C:** Przywrócenie pełnej prędkości produkcji (100%).

Dodatkowo program symuluje losowe awarie (np. zacięcie materiału), generując komunikaty w zintegrowanym dzienniku zdarzeń.

1.2 Założenia autodiagnostyki operatora (Czuwak)

W celu zapewnienia bezpieczeństwa zaimplementowano mechanizm weryfikujący "przytomność"operatora. W losowych interwałach czasowych (od 15 do 35 sekund) system wyzwala alarm wizualny. Operator ma dokładnie 30 sekund na reakcję (wciśnięcie klawisza spacji). Brak reakcji skutkuje awaryjnym wylogowaniem z systemu do ekranu startowego.

2 Opis przyjętych rozwiązań programowych

2.1 Integracja ze sprzętem (psutil)

Do pobrania parametrów pracy komputera wykorzystano zewnętrzną bibliotekę `psutil`. Zastosowano wywołanie asynchroniczne bez blokowania wątku głównego, co pozwala na płynne działanie interfejsu graficznego.

```
def update_process_parameters(self):
    # Odczyt rzeczywistego uzycia procesora
    self.cpu_usage = psutil.cpu_percent(interval=None)
    self.cpu_bar.setValue(int(self.cpu_usage))

    # Symulacja ciepla silnika na bazie obciazenia CPU
    heat_factor = (self.cpu_usage - 10) * 0.1
    cooling_factor = 5.0 if self.fan_active else 0.0

    self.simulated_temp += heat_factor - cooling_factor
    self.simulated_temp += random.uniform(-1.0, 1.0) # Szum losowy
    self.temp_bar.setValue(int(self.simulated_temp))

    self.check_limits() # Sprawdzenie stanow alarmowych
```

Listing 1: Aktualizacja parametrow na bazie sprzetu PC

2.2 Realizacja mechanizmu Czuwaka

Logika autodiagnostyki opiera się na kaskadowym wykorzystaniu obiektów `QTimer`. Jeden timer losuje moment wyzwolenia alarmu, a drugi odlicza czas na reakcję. Wychwytywanie obecności realizowane jest na poziomie głównego okna przy użyciu zdarzeń klawiatury.

```
def keyPressEvent(self, event):
    # Reakcja wylacznie na spacje podczas trwania alarmu
    if event.key() == Qt.Key.Key_Space and self.is_checking_presence:
        self.is_checking_presence = False
        self.presence_countdown_timer.stop()
        self.update_warning_banner()
        self.log_event("INFO: Obecnosc operatora potwierdzona.")
        # Zaplanowanie kolejnego sprawdzenia
        self.schedule_next_presence_check()
```

Listing 2: Weryfikacja obecnosci operatora

3 Dyskusja osiągniętych wyników (Wady i Zalety)

3.1 Zalety napisanej aplikacji

- **Realizm i responsywność:** Użycie biblioteki `psutil` pozwala na fizyczną interakcję z programem – operator może obciążyć swój komputer zewnętrznym programem (np. testem wydajności), aby zaobserwować rzeczywisty wzrost temperatury wirtualnej linii produkcyjnej.
- **Bezpieczeństwo wielopoziomowe:** Zastosowano zaawansowaną logikę kaskadową timerów w implementacji czuwaka, która gwarantuje wylogowanie nieobecnego operatora, odcinając mu dostęp do krytycznych systemów.

- **Design techniczny (SCADA UI):** Aplikacja wykorzystuje dedykowany arkusz stylów (QSS), czytelne paski postępu oraz konsolę logów z precyzyjnymi znacznikami czasu, wiernie oddając wygląd profesjonalnego oprogramowania przemysłowego.

3.2 Wady i ograniczenia rozwiązania

- **Uproszczony model termodynamiczny:** Zmiana "temperatury"realizowana jest za pomocą liniowego dodawania lub odejmowania stałych wartości skorelowanych z użyciem procesora. W rzeczywistości procesy nagrzewania i stygnięcia urządzeń przemysłowych mają charakter nieliniowy (najczęściej logarytmiczny).

- **Brak trwałej bazy danych:** System logów opiera się na komponencie `QTextEdit`. W chwili zamknięcia aplikacji (lub wylogowania), historia zdarzeń i awarii jest bezpowrotnie tracona. W profesjonalnym rozwiązaniu wymagany byłby zapis do zewnętrznej bazy (np. SQLite) lub pliku tekstowego w celu późniejszego audytu.

- **Wielowątkowość:** Choć `psutil` został wywołany w trybie nieblokującym, cała logika aplikacji wykonuje się w głównym wątku (GUI Thread). W przypadku rozbudowy aplikacji o bardziej złożone odczyty (np. bezpośrednie sprawdzanie temperatury rdzeni poprzez rejestry płyty głównej), interfejs mógłby ulegać sporadycznemu zamrażaniu.